

Threat Modeling LLM Applications

Posted by Gavin Klondike on 06 June 2023

Threat Modeling LLM Applications

Before we get started: Hi! My name is GTKlondike, and these are my opinions as a cybersecurity consultant. While experts from the AI Village provided input, I will always welcome open discussion so that we can come to a better understanding of LLM security together. If you'd like to continue this conversation, you can reach me on Twitter at [@GTKlondike](https://twitter.com/GTKlondike) (<https://twitter.com/GTKlondike>). And checkout my [YouTube channel](https://www.youtube.com/@NetsecExplained), [Netsec Explained](https://www.youtube.com/@NetsecExplained) (<https://www.youtube.com/@NetsecExplained>), for more advanced security topics.

This past week, OWASP kicked-off their [OWASP Top 10 for Large Language Model \(LLM\) Applications project](https://owasp.org/www-project-top-10-for-large-language-model-applications/) (<https://owasp.org/www-project-top-10-for-large-language-model-applications/>). I'm happy that LLM security is being taken seriously and feel fortunate to have joined the kick-off and Slack for this project.

As part of our conversations, there's been a bit of debate around what's considered a vulnerability and what's considered a feature of how LLMs operate. So I figured now would be a good time to take a stab at building a high-level threat model to suss out

these differences and contribute to a greater understanding of LLMs in a security context. I'd also like for this post to act as a starting point for anyone interested in building or deploying their own LLM applications.

Finally, we'll use this definition of a vulnerability moving forward:

`Vulnerability: A weakness in an information system, system security procedures, internal controls, or implementation that could be exploited or triggered by a threat source.`

Data Flow Diagram

Like all good threat models, we're going to start with a data flow diagram (DFD). (https://owasp.org/www-community/Threat_Modeling_Process#data-flow-diagrams). This will give us an idea of what a hypothetical application would look like, and where the associated trust boundaries should be. To keep things general, we're going to be using a level 0 DFD. In the figure below, there's only a few elements and their associated trust boundaries:

- External prompt sources (e.g., users, website content, email bodies, etc.)
- The LLM model itself (e.g., GPT4, LLaMA, Claude, etc.)
- Server-side functions (e.g., LangChain tools and components)
- Private data sources (e.g., internal documents or vector databases)

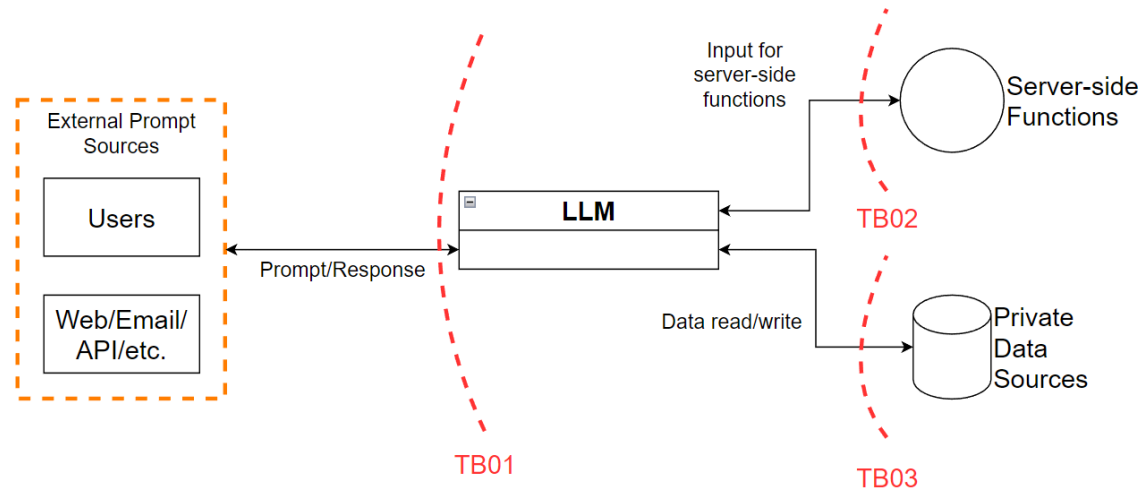


Figure 1 - Level 0 data flow diagram (DFD)

Trust boundary **TB01** is between the external endpoints and the LLM itself. In a traditional application, this would be considered the “opportunities for injection”, where an attacker could submit malicious code to manipulate an application. However, LLMs are unique in that we should not only consider the input from users as untrusted, but the output of LLMs as untrusted. The reasoning is that users and external sources could convince the LLM to modify its output in user-controlled ways. This also forces **TB01** to be a two-way trust boundary. Not only can the user modify the input, but the output can be modified to attack users. For example, returning a XSS payload back to the user but outside of a code block that would have prevented arbitrary execution (<https://packetstormsecurity.com/files/171665/ChatGPT-Cross-Site-Scripting.html>).

TB02 and TB03 MUST be considered when building applications around LLMs!

Proper controls along **TB02** would mitigate the impact of an LLM passing input directly into an `exec()` function or modifying the presentation of a XSS payload being passed back to the user. Likewise, proper controls along **TB03** would mitigate the ability for

either the LLM or an external user to gain access to sensitive data stores since, by nature, the LLMs themselves do not adhere to authorization controls internally.

Assumptions

We're also going to introduce a few assumptions we have about the hypothetical application. This is standard practice in more rigorous threat modeling exercises so we're going to do that here as well:

- This is a private or a fine-tuned model, as commonly seen in custom applications.
- The surrounding application follows standard OWASP Top 10 recommendations. Traditional application flaws are out of scope for this threat model.
- The surrounding application enforces proper application level authentication and authorization access to the LLM itself, but not between the LLM and other internal components.
- Unfettered LLM API access is a threat scenario since this is already being seen in the wild.
- For this exercise, Denial of Service (DoS/DDoS) is being ignored.
- We assume conversations are stored to debug and improve future models. This can impact user privacy, so we'll consider it.
- Content filters are not in place between the LLM and functions or data sources. This has also been commonly seen amongst more recent LLM applications.
- Prompts from server-side functions are from internal channels only. External input (e.g., website lookup or email parsing) is considered to be performed by external source entities.

STRIDE

Cool! Now that we have our DFD and assumptions written out, now we can start with the actual threat enumeration piece. This tends to be the most time-consuming piece of the exercise so do not mistake this as an exhaustive list.

If you're not familiar, STRIDE is a standard framework in threat modeling exercises. Each letter in the name corresponds to a specific threat category. This STRIDE is incredibly useful in identifying strengths and weaknesses in a system. They are as follows:

- Spoofing - A threat action aimed at accessing the user of another's credentials, such as username and password (Authentication).
- Tampering - A threat action that maliciously changes or modifies persistent data, such as database records or data in transit (Integrity).
- Repudiation - A threat action aimed at performing actions that lack the ability to be traced back to the malicious user (Non-repudiation).
- Information disclosure - A threat action with the intention of reading information that a user was not authorized to access (Confidentiality).
- Denial of Service - A threat action attempting to prevent or deny legitimate access to valid users (Availability).
- Elevation of privilege - A threat action attempting to gain privileged access to a resource, typically to further unauthorized access to additional resources (Authorization).

Strengths and Weaknesses

When doing this exercise, I find it helpful to take each trust boundary and create tables for the elements communicating across that trust boundary. Typically, I include three tables: left side element, right side element, and communication channel itself.

However, since this is a high-level threat model to get us started, two tables will be enough.

For each table we'll have a column listing out each category of STRIDE as our labels, then a strengths column and a weaknesses column. From there, we'll go down each category and list out possible vulnerabilities or strengths that would prevent a vulnerability in that category. So let's take a look at each trust boundary.

Trust Boundary: TB01

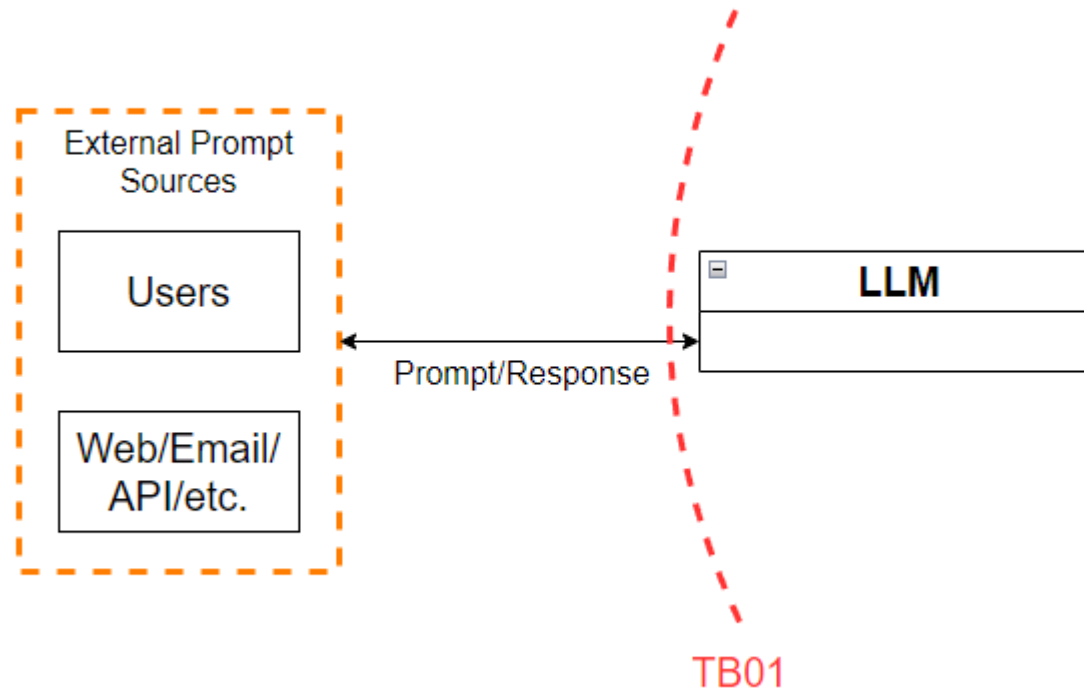


Figure 2 - Trust boundary TB01

This trust boundary lies between the users and external entities such as websites or email, and the LLM itself. As mentioned in the DFD section, **TB01** should be considered a two-way trust boundary. For that reason, we'll list out the weaknesses in controls regarding traffic from external entities as well as traffic from the LLM itself back to users.

TB01		
External Endpoints		
	Strengths	Weaknesses
Spoofing		VULN01: Modify System prompt (prompt injection)
Tampering		VULN02: Modify LLM parameters (temperature, length, model, etc.)
Repudiation	Proper authentication and authorization (assumed)	
Information Disclosure		VULN03: Input sensitive information to a third-party site (user behavior)
Denial of Service		
Elevation of Privilege	Proper authentication and authorization (assumed)	

LLM		
	Strengths	Weaknesses
Spoofing		
Tampering		
Repudiation		
Information Disclosure		VULN04: LLMs are unable to filter sensitive information (open research)
Denial of Service		
Elevation of Privilege		

Figure 3 - Trust boundary TB01 - STRIDE table

As shown above, we have four vulnerabilities right off the bat:

Vuln ID	Description	Examples
VULN01	Modify System prompt (prompt injection)	Users can modify the system-level prompt restrictions to "jailbreak" the LLM and overwrite previous controls in place
VULN02	Modify LLM parameters (temperature, length, model, etc.)	Users can modify API parameters as input to the LLM such as temperature, number of tokens returned, and model being used.
VULN03	Input sensitive information to a third-party site (user behavior)	Users may knowingly or unknowingly submit private information such as HIPAA details or trade secrets into LLMs.
VULN04	LLMs are unable to filter sensitive information (open research area)	LLMs are not able to hide sensitive information. Anything presented to an LLM can be retrieved by a user. This is an open area of research.

Table 1 - Trust boundary TB01 - vulnerability list

Trust Boundary: TB02

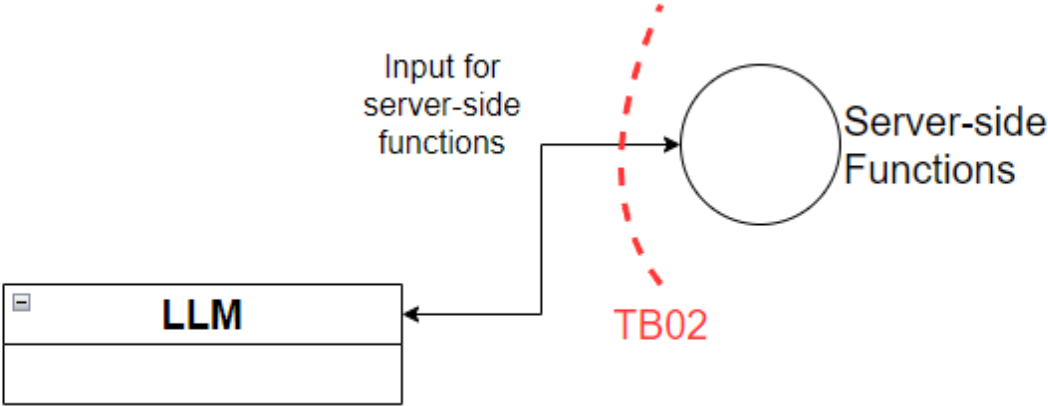


Figure 4 - Trust boundary TB02

This trust boundary lies between the LLM and backend functions or services. For example, a LLM may request a back-end function to perform some action based on a prompt it's received. In the case of **TB02**, we want to make sure we're not passing unfiltered requests from the LLM to the back-end functions. Just like we should put both client-side and server-side controls in modern web applications, we should also put client-side and server-side controls in LLM applications.

TB02	LLM	
	Strengths	Weaknesses
Spoofing		VULN05: Output controlled by prompt input (unfiltered)
Tampering		VULN05: Output controlled by prompt input (unfiltered)
Repudiation		
Information Disclosure		
Denial of Service		
Elevation of Privilege		

Server-side Functions		
	Strengths	Weaknesses
Spoofing	Server-side functions maintain separate access to LLM from users	
Tampering		VULN06: Server-side output can be fed directly back into LLM (requires filter)
Repudiation		
Information Disclosure		VULN06: Server-side output can be fed directly back into LLM (requires filter)
Denial of Service		
Elevation of Privilege		

Figure 5 - Trust boundary TB02 - STRIDE table

Two major vulnerabilities stick out to us:

Vuln ID	Description	Examples
VULN05	Output controlled by prompt input (unfiltered)	LLM output can be controlled by users and external entities. Unfiltered acceptance of LLM output could lead to unintended code execution.
VULN06	Server-side output can be fed directly back into LLM (requires filter)	Unrestricted input to server-side functions can result in sensitive information disclosure or server-side request forgery (SSRF). Server-side controls would mitigate this impact.

Table 2 - Trust boundary TB02 - vulnerability list

Trust Boundary: TB03

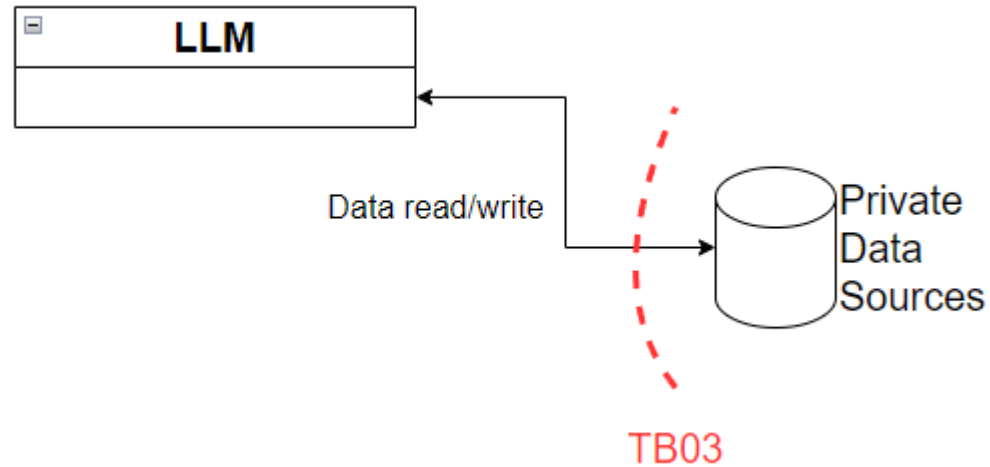


Figure 6 - Trust boundary TB03

This is the last trust boundary we're going to take a look at, **TB03**. This trust boundary lies between the LLM and any private data stores one might have. For example, reference documentation, internal websites, private databases, etc. In here, we'll want to ensure that we are applying necessary authorization controls and the principle of least privilege outside of the LLM since LLMs themselves cannot do this on their own.

TB03		LLM
	Strengths	Weaknesses
Spoofing		VULN05: Output controlled by prompt input (unfiltered)
Tampering		VULN05: Output controlled by prompt input (unfiltered)
Repudiation		
Information Disclosure		
Denial of Service		
Elevation of Privilege		

Private Data Sources		
	Strengths	Weaknesses
Spoofing		
Tampering		
Repudiation		
Information Disclosure		VULN07: Access to sensitive information
Denial of Service		
Elevation of Privilege		

Figure 7 - Trust boundary TB03 - STRIDE table

As one would expect, we see one familiar vulnerability along with a new one on the private data sources element:

Vuln ID	Description	Examples
VULN05	Output controlled by prompt input (unfiltered)	LLM output can be controlled by users and external entities. Unfiltered acceptance of LLM output could lead to unintended code execution.
VULN07	Access to sensitive information	LLMs have no concept of authorization or confidentiality. Unrestricted access to private data stores would allow users to retrieve sensitive information.

Table 3 - Trust boundary TB03 - vulnerability table

Things not on this List

What about hallucinations?

If you’ve followed AI in the news lately, I’m sure you’ve come across the term “hallucinations” to describe when LLMs completely fabricate facts as part of their output (<https://www.cnn.com/2023/05/27/business/chat-gpt-avianca-mata-lawyers/index.html>). Many would consider this to be a vulnerability, but I’d push back on that. When you look at what exactly LLMs do to generate their output, they use previous tokens to predict the statistically most likely token to come next. You’re already familiar with this approach if you’ve ever used the predictive text feature in your smartphone: “Finish this sentence with the most likely answer, ‘the cat sat on the ____.’” Therefore, this is not a bug, it’s a feature and as security professionals we should plan around this known limitation.

One last thing to say here is that the term, “hallucination” is being disputed in AI circles. Hallucinations are part of a human psychological experience, when in this case the LLMs are instead just confidently making things up. Another proposed term is “confabulation” (<https://www.bereni.io/2023-03-19-LLMs-confabulate-not-hallucinate/>).

Personally, I just say they make things up. Public communication is hard enough, don't let words get in the way of the message.

What about training data poisoning, bias, or hate speech?

This is a bit of a personal thought: Yes, data poisoning, bias, and hate speech is a problem in the development and fine-tuning of the model itself. At the same time, this application-level threat model assumes that the LLM is complete already.

With that, for data poisoning to occur, likely 1 of 2 things would have to happen:

1. Attacker gains engineer/developer level access to the training data to cause modification. (Similar to gaining access to the CI/CD pipeline in modern applications).
2. Engineers/developers accept training data directly from untrusted sources (e.g., Microsoft Tay AI from 10 years ago).

I think to accept this as a risk worthy of our list, we should locate a recent CVE or real-world example of this occurring outside of a purely academic space. I'm not saying it isn't possible, I'm just not convinced it's common enough to warrant being on the list.

Recommendations

To round out this exercise, here are some high-level recommendations to get us started. As mentioned at the beginning, this is not an exhaustive list.

VULN01 - Modify System prompt (prompt injection)

Not a lot we can do directly, but we can mitigate the downstream effects. Ensure the LLM isn't trained on non-public or confidential data. Additionally, treat all LLM output as untrusted and enforce necessary restrictions to data or actions the LLM requests.

VULN02 - Modify LLM parameters (temperature, length, model, etc.)

Limit the API attack surface that is exposed to external prompt sources (users, website content, email bodies, etc.). As always, treat external input as untrusted and apply filtering where appropriate.

VULN03 - Input sensitive information to a third-party site (user behavior)

This is a user behavior problem but we can educate users via statements presented during the signup process and through clear, consistent notifications every time a user connects to the LLM.

VULN04 - LLMs are unable to filter sensitive information (open research area)

Do not train LLMs on sensitive data that all users should not have access to. Additionally, do not rely on LLMs to enforce authorization controls to data sources. Instead, apply those controls to the data sources themselves.

VULN05 - Output controlled by prompt input (unfiltered)

Treat all LLM output as untrusted and apply appropriate restrictions prior to using results as input to additional functions. This will mitigate the impact a maliciously crafted prompt could have on functions and services within the internal environment.

VULN06 - Server-side output can be fed directly back into LLM (requires filter)

Perform appropriate filtering on server-side function output. If the LLM stores the output for future training data, ensure sensitive information is sanitized prior to retraining. If the LLM returns function output (in any form) back to a user, ensure sensitive information is removed prior to returning such information.

VULN07 - Access to sensitive information

Treat access from the LLM as though it were a typical user. Enforce standard authentication/authorization controls prior to data access. LLMs have no way to protect sensitive information from unauthorized users, so the controls must be placed here.

Summary

Altogether, this exercise has been pretty fruitful. We have 7 high-level vulnerabilities to address. Again, this is just to get us started by walking through the process of formally threat modeling an LLM application. Here's the full table of the vulnerabilities we've identified so far, in the order we that found them above:

Vuln ID	Description	Examples
VULN01	Modify System prompt (prompt injection)	Users can modify the system-level prompt restrictions to "jailbreak" the LLM and overwrite previous controls in place.
VULN02	Modify LLM parameters (temperature, length, model, etc.)	Users can modify API parameters as input to the LLM such as temperature, number of tokens returned, and model being used.
VULN03	Input sensitive information to a third-party site (user behavior)	Users may knowingly or unknowingly submit private information such as HIPAA details or trade secrets into LLMs.
VULN04	LLMs are unable to filter sensitive information (open research area)	LLMs are not able to hide sensitive information. Anything presented to an LLM can be retrieved by a user. This is an open area of research.
VULN05	Output controlled by prompt input (unfiltered)	LLM output can be controlled by users and external entities. Unfiltered acceptance of LLM output could lead to unintended code execution.
VULN06	Server-side output can be fed directly back into LLM (requires filter)	Unrestricted input to server-side functions can result in sensitive information disclosure or SSRF. Server-side controls would mitigate this impact.
VULN07	Access to sensitive information	LLMs have no concept of authorization or confidentiality. Unrestricted access to private data stores would allow users to retrieve sensitive information.

Table 4 - Full vulnerability list

And a list of recommendations to help mitigate each vulnerability:

REC ID	Recommendations
REC01	Not a lot we can do directly, but we can mitigate the downstream effects. Ensure the LLM isn't trained on non-public or confidential data. Additionally, treat all LLM output as untrusted and enforce necessary restrictions to data or actions the LLM requests.
REC02	Limit the API attack surface that is exposed to external prompt sources (users, website content, email bodies, etc.). As always, treat external input as untrusted and apply filtering where appropriate.
REC03	This is a user behavior problem but we can educate users via statements presented during the signup process and through clear, consistent notifications every time a user connects to the LLM.
REC04	Do not train LLMs on sensitive data that all users should not have access to. Additionally, do not rely on LLMs to enforce authorization controls to data sources. Instead, apply those controls to the data sources themselves.
REC05	Treat all LLM output as untrusted and apply appropriate restrictions prior to using results as input to additional functions. This will mitigate the impact a maliciously crafted prompt could have on functions and services within the internal environment.
REC06	Perform appropriate filtering on server-side function output. If the LLM stores the output for future training data, ensure sensitive information is sanitized prior to retraining. If the LLM returns function output (in any form) back to a user, ensure sensitive information is removed prior to returning such information.
REC07	Treat access from the LLM as though it were a typical user. Enforce standard authentication/authorization controls prior to data access. LLMs have no way to protect sensitive information from unauthorized users, so the controls must be placed here.

Table 5 - Full recommendations list

That’s a wrap! I hope this exercise has been helpful and that it’s empowered you to be more security conscious when it comes to building and deploying LLM powered applications.

2024	1	2023	5	2022	7
2018	3				

2024

AI Village Announcing Generative Red Team 2 at DEF CON 32

🕒 5 minute read

[BACK TO TOP ↑](#)

2023

Generative Red Team Recap

🕒 27 minute read

Generative Red Team History

Threat Modeling LLM Applications

🕒 19 minute read

Threat Modeling LLM Applications Before we get started: Hi! My name is GTKlondike, and these are my opinions as a cybersecurity consultant. While experts fr...

AI Village at DEF CON announces largest-ever public Generative AI Red Team

🕒 5 minute read

Largest annual hacker convention to host thousands to find bugs in large language models built by Anthropic, Google, Hugging Face, NVIDIA, OpenAI, and Stabli...

The Spherical Cow of ML Security

🕒 10 minute read

The Spherical Cow of Machine Learning Security

Prompt Detective at SXSW!

🕒 1 minute read

Prompt Detective Announcement

[BACK TO TOP ↑](#)

2022

The AI RMF Does not Address Common Needs

🕒 10 minute read

Disclaimer: This does not reflect the AIV as a whole, these are my opinions and this was my response.

AI and Hiring Tech Panel

🕒 4 minute read

AI and ML is already being used to identify job candidates, screen resumes, assess worker productivity and even help tag candidates for firing. Can the inter...

The Use of AI/ML in Offensive Security Operations

🕒 3 minute read

The Red Team Village and the AI Village will host a panel from different industry experts to discuss the use of artificial intelligence and machine learning ...

DEFCON 30 Sunday Schedule

🕒 4 minute read

Automate Detection with Machine Learning

DEFCON 30 Saturday Schedule

🕒 7 minute read

A few useful things to know about AI Red Teams

DEFCON 30 Friday Schedule

🕒 8 minute read

Automate Detection with Machine Learning

Generative Art

🕒 5 minute read

Generative Art at AI Village DEF CON 30

[BACK TO TOP ↑](#)

2018

Gradient Attacks

🕒 11 minute read

Welcome to the second post in the AI Village's adversarial machine learning series. This one will cover the greedy fast methods that are most commonly used. ...

Max evil MLsec, why should you care?

🕒 7 minute read

Originally posted on Medium - follow @sarajayneterp and like her article there

Dimensionality and Adversarial Examples

🕒 11 minute read

Welcome to AI Village's series on adversarial examples. This will focus on image classification attacks as they are simpler to work with and this series is m...

[BACK TO TOP ↑](#)